Packet Operator Algorithms, Exact Compilation, and GPU Realization

By Komorebi P.

Abstract

A packet operator is a linear map written between the members of a finite direct sum. We study the resulting computational representation before choosing a processor or kernel. Exact formulas are given for block sparse application, block diagonal plus low rank algebra, Woodbury inversion, support propagation, block preconditioning, and packetwise perturbation. Products of low rank corrections admit optimal recompression through a small singular value problem, with explicit error propagation through a product chain. A rooted candidate decomposition supports an exact compiler when representation cost and error are additive. Dynamic programming gives the optimal Lagrangian pruning, an exact quantized error budget algorithm, and a local update rule. Discrete precision assignment becomes a multiple choice budget problem. Arithmetic, storage, data movement, launch count, discovery, transfer, and materialization are kept as separate costs. A sanitized corpus of 154 recorded CPU and GPU measurements illustrates regular packet gains, precision frontiers, crossover behavior, and retained negative controls. The largest matched structured result is a factor of 14.89 against a device resident dense GPU calculation. The paper proves finite algebraic and optimization statements. Hardware measurements remain specific to their recorded environment, and no generic dense matrix exponent or universal discovery algorithm is claimed.

1. Introduction

A matrix can look dense while its action is not. Independent diagonal blocks, sparse transfers, low rank couplings, tensor factors, and transform coordinates can remove most of the arithmetic and storage before a kernel is launched. The gain is mathematical first. A device can exploit only the structure that the representation exposes.

Keywords: structured matrix, low rank update, dynamic programming, mixed precision, GPU computation, benchmark contract

AMS Classification: Primary: 65F30, Secondary: 65F50, 65Y05, 68W10, 68W40.

This article begins with a finite direct sum

$$X = \bigoplus_{\alpha \in I} X_\alpha$$

and a complete family of coordinate projectors

$$P_\alpha P_\beta = \delta_{\alpha\beta} P_\alpha, \qquad \sum_{\alpha \in I} P_\alpha = I.$$

An operator has packet blocks

$$A_{\beta\alpha} = P_\beta A P_\alpha.$$

The active addresses form a directed graph. Block dimensions, block types, approximation tolerances, and output requirements turn this graph into a computational intermediate representation.

The underlying linear algebra is classical. Direct and iterative methods are treated in [14], [24], and [9]. Low rank approximation and updates are governed by singular value decomposition and the Sherman Morrison Woodbury identity [11], [21], [15], and [4]. Floating point analysis follows the standard model in [17] and [19]. Randomized discovery of low rank structure is developed in [16] and [20]. Communication and attainable performance are shaped by data movement and arithmetic intensity [18], [26], [2], and [12].

The first task is semantic. A compressed representation must specify the operator that it denotes. It must also specify what an algorithm returns. A product represented by block and low rank factors is not the same output as an explicit matrix. A timing comparison is valid only after the output contract and timing scope agree.

The second task is algebraic. Block diagonal plus low rank operators are closed under multiplication before recompression. Their unrecompressed rank grows additively. Woodbury inversion reduces a large inverse to packet solves and one small dense inverse. The low rank factors can then be recompressed by orthogonalizing them and taking the singular value decomposition of a small core. This produces the best truncated approximation in spectral and Frobenius norm.

The third task is combinatorial. Sparse transfer graphs predict possible support under powers, polynomials, and Schur elimination. A directed acyclic graph gives a finite path expansion. A block triangular solve can be scheduled by topological level. Graph regularity matters because equal shapes can be batched, while irregular edges require indexing and accumulation.

The fourth task is numerical. Packet perturbations can be certified by their block sensitivities. A continuous model assigns tolerances through a convex calculation. Upper bounds on available tolerances produce a clipped allocation.

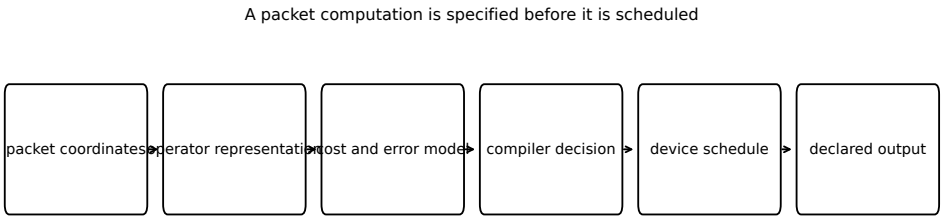


Figure 1. The representation and the output contract are fixed before the device schedule is chosen.

A finite set of data types gives a multiple choice budget problem that can be solved exactly after the errors are quantized.

The central computational result concerns compilation. Suppose a finite rooted tree records candidate refinements. Each node may be kept as one representation or split into its children. If cost and error are additive across the selected leaves, the optimal pruning is found exactly by dynamic programming. A quantized error budget has a second exact dynamic program. The decision is stable when the score gap exceeds the perturbation in the local models. A change inside one subtree requires recomputation only along the ancestor chain. These results prove a restricted packet compiler. They do not solve unrestricted discovery or arbitrary device scheduling.

The measured part uses a sanitized corpus collected on one recorded CPU and one NVIDIA GeForce RTX 3070 Ti Laptop GPU. The corpus contains 154 measured rows from sixteen suites. Regular equal block operators and block diagonal plus low rank operators show large gains. Recursive HODLR, Strassen multiplication, iterative refinement, a small Walsh Hadamard transform, and an attempted two stream overlap provide negative controls. These rows are retained because structure does not guarantee speed.

The article separates four sources of gain:

$$\text{representation}, \qquad \text{device}, \qquad \text{kernel}, \qquad \text{amortization}.$$

A single speedup number can mix all four. The benchmark protocol therefore records case identity, output contract, timing scope, source precision, correctness reference, discovery time, compilation time, and reuse count.

The mathematical statements are finite. The empirical statements apply only to the recorded environment and generated cases. The article does not claim a new generic matrix multiplication exponent. Fast matrix multiplication and its stability form a separate theory [25] and [10]. It also does not claim that every structured matrix has a profitable GPU realization.

2. **Packet operator representations**

2.1. *Exact semantics.*

Definition 2.1 (Packet layout). A packet layout is a finite family of finite dimensional Hilbert spaces

$$\mathcal{P} = (X_\alpha)_{\alpha \in I}$$

with direct sum

$$X = \bigoplus_{\alpha \in I} X_\alpha.$$

Write

$$n_\alpha = \dim X_\alpha, \qquad n = \sum_{\alpha \in I} n_\alpha.$$

The coordinate projector onto X_α is denoted by P_α.

Definition 2.2 (Packet operator representation). A finite packet operator representation consists of

$$\mathsf{IR} = (\mathcal{P}, E, (B_{\beta\alpha})_{(\beta,\alpha)\in E}),$$

where

$$E \subseteq I \times I$$

and

$$B_{\beta\alpha} : X_\alpha \longrightarrow X_\beta.$$

Its evaluation is

$$\operatorname{eval}(\mathsf{IR}) = \sum_{(\beta,\alpha)\in E} \iota_\beta B_{\beta\alpha} P_\alpha,$$

where $\iota_\beta : X_\beta \to X$ is the coordinate inclusion.

Theorem 2.3 (Exact evaluation). *Let $A \in \operatorname{End}(X)$. Set*

$$B_{\beta\alpha} = P_\beta A P_\alpha$$

and let E contain exactly the nonzero blocks. Then

$$\boxed{A = \sum_{(\beta,\alpha)\in E} P_\beta A P_\alpha.}$$

For every matrix of right hand sides Z,

$$AZ = \sum_{(\beta,\alpha)\in E} B_{\beta\alpha} P_\alpha Z.$$

Proof. Insert the identity on both sides of A:

$$A = \left(\sum_\beta P_\beta \right) A \left(\sum_\alpha P_\alpha \right) = \sum_{\alpha,\beta} P_\beta A P_\alpha.$$

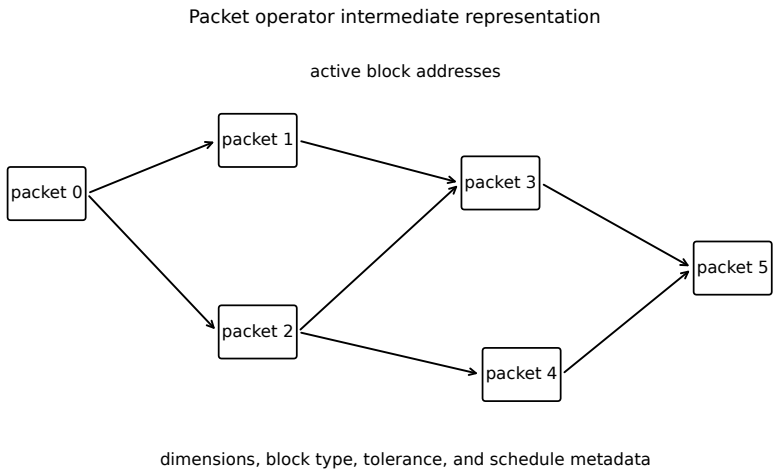


Figure 2. A packet intermediate representation stores active block addresses together with dimensions, block types, tolerances, and scheduling metadata.

The zero blocks can be removed. Applying the identity to Z gives the second formula. □

Definition 2.4 (Support graph). The support graph G_A has vertex set I. It has a directed edge

$$\alpha \longrightarrow \beta$$

when $P_\beta A P_\alpha \neq 0$.

THEOREM 2.5 (Block composition). *Let* $A, B \in \mathrm{End}(X)$. *Then*

$$\boxed{(AB)_{\gamma\alpha} = \sum_{\beta \in I} A_{\gamma\beta} B_{\beta\alpha}.}$$

Consequently,

$$\mathrm{supp}(AB) \subseteq \mathrm{supp}(A) \circ \mathrm{supp}(B),$$

where relational composition is defined by

$$(\gamma, \alpha) \in E \circ F$$

when some β *satisfies*

$$(\gamma, \beta) \in E, \qquad (\beta, \alpha) \in F.$$

Proof. Insert the packet identity between A and B:

$$P_\gamma A B P_\alpha = \sum_\beta P_\gamma A P_\beta B P_\alpha.$$

A summand can be nonzero only when both block addresses are active. □

COROLLARY 2.6 (Adjoint and tensor semantics). *The adjoint representation has blocks*

$$(A^*)_{\alpha\beta} = A^*_{\beta\alpha}.$$

If A and B are packet resolved on X and Y, then

$$(A \otimes B)_{(\beta,\delta),(\alpha,\gamma)} = A_{\beta\alpha} \otimes B_{\delta\gamma}.$$

Proof. Both identities follow by inserting the corresponding coordinate projectors and using

$$(P_\beta A P_\alpha)^* = P_\alpha A^* P_\beta.$$

The tensor identity follows from

$$(P_\beta \otimes Q_\delta)(A \otimes B)(P_\alpha \otimes Q_\gamma) = A_{\beta\alpha} \otimes B_{\delta\gamma}.$$

□

2.2. *Output contracts.*

Definition 2.7 (Output contract). An output contract specifies the mathematical object returned by an algorithm. The contract may require an explicit vector, an explicit matrix, block factors, low rank factors, spectral factors, or an implicit operator with a declared application rule.

Definition 2.8 (Timing scope). A timing scope states which costs are included. Possible costs include structure discovery, compilation, host transfer, device transfer, kernel execution, graph capture, graph replay, factor reconstruction, and explicit materialization.

PROPOSITION 2.9 (Contract conversion). *Suppose one algorithm returns a compressed representation R and another returns an explicit object Y. A direct timing comparison is valid only if*

$$\mathrm{decode}(R) = Y$$

and the time required by decode *is included, or if the downstream task consumes R directly.*

Proof. Without decoding, the two procedures return different mathematical objects. A timing ratio between different outputs does not compare implementations of the same map. Including the decoding cost gives two procedures with the same output. If the next operation consumes R, decoding is not part of the declared computation. □

Remark 2.10. A compressed product and an explicit matrix product may both be exact. They still have different contracts. This distinction becomes decisive when materialization costs n^2 entries.

3. **Arithmetic, storage, and data movement**

We count one scalar multiplication and one scalar addition as two modeled floating point operations. These counts describe a representation. They do not predict wall time without a memory and launch model.

THEOREM 3.1 (Dense and packet application counts). *A dense n by n matrix applied to q right hand sides has modeled cost*

$$\boxed{\mathrm{flop}_{\mathrm{dense}} = 2n^2 q.}$$

A packet representation with active edge set E has modeled cost

$$\boxed{\mathrm{flop}_{\mathrm{packet}} = 2q \sum_{(\beta,\alpha)\in E} n_\beta n_\alpha.}$$

It stores

$$\boxed{M_{\mathrm{packet}} = \sum_{(\beta,\alpha)\in E} n_\beta n_\alpha}$$

scalar entries.

Proof. The active block $B_{\beta\alpha}$ is an n_β by n_α matrix. Applying it to an n_α by q input costs $2n_\beta n_\alpha q$ modeled operations and stores $n_\beta n_\alpha$ scalars. Summation over the active addresses gives both formulas. □

COROLLARY 3.2 (Equal diagonal packets). *Suppose n is divided into equal diagonal packets of width b. Then*

$$\mathrm{flop}_{\mathrm{block}} = 2qnb, \qquad M_{\mathrm{block}} = nb.$$

The dense to block reduction factor is

$$\boxed{\frac{n}{b}.}$$

Proof. There are n/b active diagonal blocks. Each contains b^2 entries. □

Definition 3.3 (Block diagonal plus low rank operator). A block diagonal plus low rank operator has the form

$$A = D + UV^*,$$

where

$$D = \bigoplus_{\alpha\in I} D_\alpha, \qquad U, V \in \mathbb{C}^{n\times r}.$$

The abbreviation BDLR will be used below.

THEOREM 3.4 (BDLR application and storage). *For q right hand sides,*

$$AZ = DZ + U(V^*Z)$$

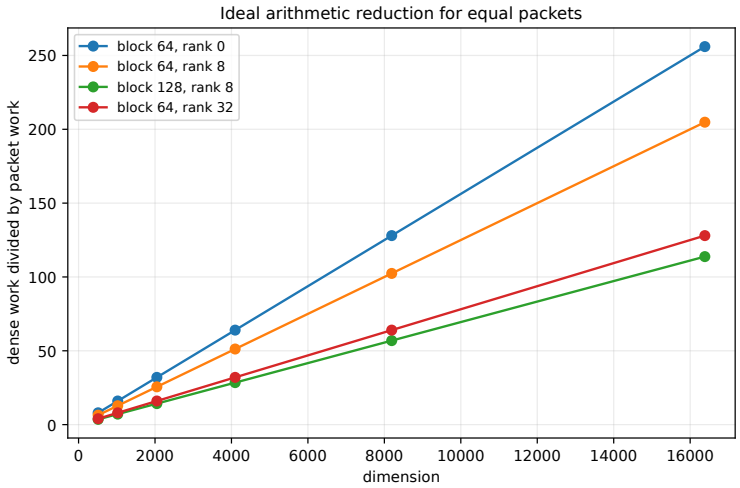


Figure 3. The ideal reduction grows with dimension only when packet width and transfer rank remain controlled.

has modeled cost

$$\boxed{2q\left(\sum_{\alpha} n_{\alpha}^{2} + 2nr\right).}$$

The representation stores

$$\boxed{\sum_{\alpha} n_{\alpha}^{2} + 2nr}$$

scalars. For equal packets of width b*, both ideal dense reduction factors equal*

$$\boxed{\frac{n}{b+2r}.}$$

Proof. The block diagonal product costs

$$2q\sum_{\alpha} n_{\alpha}^{2}.$$

The products $V^{*}Z$ and $U(V^{*}Z)$ cost $2nrq$ each. Storage consists of the diagonal blocks and the two low rank factors. For equal packets,

$$\sum_{\alpha} n_{\alpha}^{2} = nb.$$

□

Definition 3.5 (One pass streaming model). Fix a scalar size s bytes. For every active block, the one pass streaming model reads the block once, reads its source packet once, and reads and writes its target packet once. It does not assume cache reuse across edges.

PROPOSITION 3.6 (Streaming byte count). *Under definition* 3.5, *applying a packet operator to q right hand sides moves*

$$\boxed{B_{\text{stream}} = s \sum_{(\beta,\alpha)\in E} (n_\beta n_\alpha + n_\alpha q + 2n_\beta q)}$$

bytes.

Proof. The three terms count the matrix block, the source values, and the read plus write traffic for the target accumulation. The model counts these quantities independently for every active edge. □

Definition 3.7 (Arithmetic intensity). The modeled arithmetic intensity is

$$\mathcal{I} = \frac{\text{flop}}{\text{bytes}}.$$

THEOREM 3.8 (Roofline lower bound). *Suppose a device sustains at most P modeled operations per second and at most W bytes per second. A computation that requires F operations and B bytes satisfies*

$$\boxed{T \geq \max\left\{\frac{F}{P}, \frac{B}{W}\right\}.}$$

Equivalently, its work rate is at most

$$\min\{P, W\mathcal{I}\}.$$

Proof. The operation capacity gives $T \geq F/P$. The bandwidth capacity gives $T \geq B/W$. Both restrictions hold at once. □

The roofline model is a guide rather than an exact execution law [26]. Packet structure changes both F and B. An irregular representation may remove arithmetic while lowering intensity enough to become bandwidth or launch limited.

COROLLARY 3.9 (Capacity threshold). *Let a device have available memory M bytes and scalar size s. The matrix representation alone can fit only when*

$$n \leq \sqrt{M/s}.$$

The equal packet BDLR representation alone fits when

$$n(b + 2r)s \leq M.$$

Proof. Substitute the dense and BDLR storage counts into the capacity inequality. □

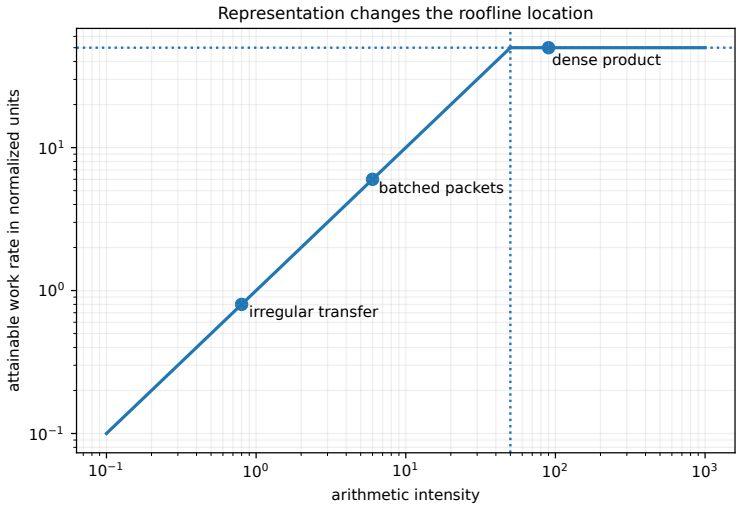


Figure 4. A representation can move the same mathematical problem between memory limited and compute limited regimes.

PROPOSITION 3.10 (Materialization cost). *Materializing $D + UV^*$ as an explicit matrix requires storage for n^2 scalars and modeled arithmetic*

$$2n^2r + O(n^2)$$

when the low rank correction is formed by dense multiplication.

Proof. The product UV^* multiplies an n by r matrix by an r by n matrix. Adding the diagonal part changes only the lower order term. □

4. **Structured packet primitives**

The packet representation can store different exact local operator types. This section records the finite formulas used by several measured suites. The results concern representation and arithmetic. They do not assert that one implementation is fastest at every size.

PROPOSITION 4.1 (Equal block banded count). *Let $p = n/b$ equal packets have width b. Suppose a block is active precisely when*

$$|i - j| \le w, \qquad 0 \le w \le p - 1.$$

The number of active blocks is

$$\boxed{N_{p,w} = p(2w + 1) - w(w + 1).}$$

For q right hand sides, the modeled application cost and scalar storage are

$$\boxed{2qb^2N_{p,w} \quad \textit{and} \quad b^2N_{p,w}.}$$

Proof. The diagonal contributes p blocks. At offset k, where $1 \le k \le w$, the upper and lower block diagonals contribute $2(p-k)$ blocks. Therefore

$$p + 2\sum_{k=1}^{w}(p-k) = p(2w+1) - w(w+1).$$

The cost and storage follow from theorem 3.1. □

PROPOSITION 4.2 (Sparse plus low rank application). *Let*

$$A = S + UV^*,$$

where S has z stored scalar nonzeros and $U, V \in \mathbb{C}^{n\times r}$. In the model that charges two operations per stored sparse entry and right hand side, applying A to q right hand sides costs

$$\boxed{2q(z+2nr)}$$

modeled operations and stores

$$\boxed{z+2nr}$$

scalar values, apart from sparse index metadata.

Proof. The sparse product costs $2zq$. The two dense thin products cost $2nrq$ each. The stored values are the nonzeros of S and the entries of the two factors. □

THEOREM 4.3 (Circulant spectral representation). *Let $c \in \mathbb{C}^n$, with indices taken modulo n, and define*

$$C(c)_{jk} = c_{j-k}.$$

Let F be the unitary discrete Fourier matrix

$$F_{jk} = \frac{1}{\sqrt{n}} \exp\left(-\frac{2\pi \mathrm{i} jk}{n}\right).$$

Then

$$\boxed{C(c) = F^* \operatorname{diag}(\sqrt{n}\, Fc) F.}$$

The operator is determined by n scalars and can be applied through two Fourier transforms and one pointwise product.

Proof. For each frequency k, let

$$v_j^{(k)} = \frac{1}{\sqrt{n}} \exp\left(\frac{2\pi \mathrm{i} jk}{n}\right).$$

Then

$$(C(c)v^{(k)})_j = \sum_{r=0}^{n-1} c_r v_{j-r}^{(k)} = \left(\sum_{r=0}^{n-1} c_r \exp\left(-\frac{2\pi \mathrm{i} rk}{n}\right)\right) v_j^{(k)}.$$

The eigenvalue is $\sqrt{n}(Fc)_k$. The vectors $v^{(k)}$ are the columns of F^*, which gives the formula. □

The fast Fourier transform evaluates the two transforms in $O(n \log n)$ arithmetic for the usual composite lengths [7]. This is an algorithmic factorization of the exact identity in theorem 4.3.

COROLLARY 4.4 (Toeplitz circulant embedding). *Let $T \in \mathbb{C}^{n\times n}$ be Toeplitz, with first column*

$$(c_0, c_1, \ldots, c_{n-1})^T$$

and first row

$$(c_0, r_1, \ldots, r_{n-1}).$$

Set

$$g = (c_0, c_1, \ldots, c_{n-1}, 0, r_{n-1}, \ldots, r_1)^T \in \mathbb{C}^{2n}$$

and append n zeros to $x \in \mathbb{C}^n$. The first n entries of the circulant product $C(g)(x, 0)^T$ equal Tx.

Proof. For $0 \le j < n$, the circular convolution sum separates into indices $k \le j$, where the coefficient is c_{j-k}, and indices $k > j$, where the wrapped coefficient is r_{k-j}. The padded coordinates do not contribute. The resulting sum is exactly the jth Toeplitz row applied to x. □

THEOREM 4.5 (Kronecker modewise application). *Let $A \in \mathbb{C}^{m\times m}$, $B \in \mathbb{C}^{n\times n}$, and $X \in \mathbb{C}^{n\times m}$. With column major vectorization,*

$$\boxed{(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(BXA^T).}$$

The right side has modeled cost

$$\boxed{2n^2m + 2nm^2}$$

and stores $n^2 + m^2$ factor entries instead of n^2m^2 entries.

Proof. The (i, j) entry of BXA^T is

$$\sum_{k=1}^{n}\sum_{\ell=1}^{m} B_{ik}X_{k\ell}A_{j\ell}.$$

This is the coordinate formula for $(A \otimes B)\,\mathrm{vec}(X)$. Multiplying first by B and then by A^T gives the stated count. □

COROLLARY 4.6 (Equal tensor factors). *A product of k square factors of order d, acting on a tensor with d^k entries, can be applied by k modewise contractions with modeled cost*

$$\boxed{2kd^{k+1}}$$

and factor storage kd^2.

Proof. For one mode, there are d^{k-1} fibers of length d. Multiplying each fiber by a d by d matrix costs $2d^2$, so one mode costs $2d^{k+1}$. Sum over the k modes. □

THEOREM 4.7 (Walsh Hadamard recursion). *Let $N = 2^m$, and define*

$$H_1 = [1], \qquad H_{2N} = \begin{bmatrix} H_N & H_N \\ H_N & -H_N \end{bmatrix}.$$

Then

$$H_N H_N^T = NI.$$

The product $H_N x$ can be evaluated in exactly $N \log_2 N$ additions or subtractions through the recursion.

Proof. The orthogonality identity follows by induction from block multiplication. One recursive stage forms $x_0 + x_1$ and $x_0 - x_1$, which uses N additions or subtractions, and then applies two transforms of size $N/2$. Thus

$$T(N) = 2T(N/2) + N, \qquad T(1) = 0,$$

whose solution is $T(N) = N \log_2 N$. □

5. **Exact low rank algebra and recompression**

5.1. *Products and inverses.*

THEOREM 5.1 (Exact BDLR product). *Let*

$$A = D_A + U_A V_A^*, \qquad B = D_B + U_B V_B^*$$

use the same diagonal packet layout. Then

$$AB = D_A D_B + U_C V_C^*,$$

where

$$U_C = \begin{bmatrix} D_A U_B + U_A (V_A^* U_B) & U_A \end{bmatrix}$$

and

$$V_C = \begin{bmatrix} V_B & D_B^* V_A \end{bmatrix}.$$

The unrecompressed correction rank is at most $r_A + r_B$.

Proof. Expand the product:

$$AB = D_A D_B + D_A U_B V_B^* + U_A V_A^* D_B + U_A (V_A^* U_B) V_B^*.$$

The second and fourth terms share the right factor V_B^*. The third term equals

$$U_A (D_B^* V_A)^*.$$

Concatenation gives the stated factors. □

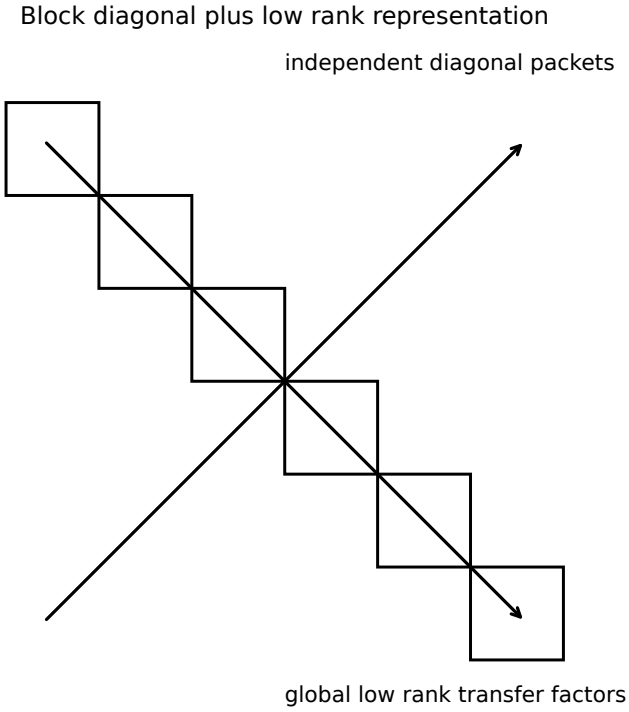


Figure 5. A regular diagonal packet structure is coupled by global low rank factors.

COROLLARY 5.2 (Rank growth in a product chain). *Let*

$$A_j = D_j + U_j V_j^*, \qquad j = 1, \dots, m,$$

share one diagonal packet layout. Before recompression, the product

$$A_m \cdots A_1$$

has a BDLR representation whose correction rank is at most

$$\boxed{\sum_{j=1}^{m} r_j.}$$

Proof. Apply theorem 5.1 inductively. Each new factor adds at most its own correction rank. □

THEOREM 5.3 (Woodbury inversion). *Assume D and*

$$S = I + V^* D^{-1} U$$

are invertible. Then

$$\boxed{(D+UV^*)^{-1} = D^{-1} - D^{-1}US^{-1}V^*D^{-1}.}$$

Proof. Multiply the proposed inverse by $D+UV^*$. The coefficient of V^*D^{-1} is

$$U - US^{-1}(I + V^*D^{-1}U) = 0.$$

The remaining term is the identity. The right inverse calculation is the same. □

The formula and its history are discussed in [15]. It reduces one large solve to packet solves with D and one dense solve with the small matrix S.

COROLLARY 5.4 (Inverse norm bound). *Under the hypotheses of theorem* 5.3,

$$\left\|(D+UV^*)^{-1}\right\| \le \left\|D^{-1}\right\| + \left\|D^{-1}U\right\| \left\|S^{-1}\right\| \left\|V^*D^{-1}\right\|.$$

Proof. Apply the triangle inequality and submultiplicativity to the Woodbury formula. □

5.2. *Optimal factor recompression.*

Standard Fact 5.5 (Eckart Young Mirsky theorem). If M has singular values

$$\sigma_1 \ge \sigma_2 \ge \cdots,$$

then its truncated singular value decomposition M_k is a best rank at most k approximation. It satisfies

$$\|M - M_k\|_2 = \sigma_{k+1}$$

and

$$\|M - M_k\|_{\mathrm{F}} = \left(\sum_{j>k} \sigma_j^2\right)^{1/2}.$$

This is classical [11] and [21]. The next theorem shows that an n dimensional singular value decomposition is unnecessary when a correction is already factored.

THEOREM 5.6 (Small core recompression). *Let*

$$L \in \mathbb{C}^{m\times r}, \qquad R \in \mathbb{C}^{n\times r}.$$

Take compact QR factorizations

$$L = Q_L R_L, \qquad R = Q_R R_R,$$

where the columns of Q_L and Q_R are orthonormal. Form the small core

$$G = R_L R_R^*$$

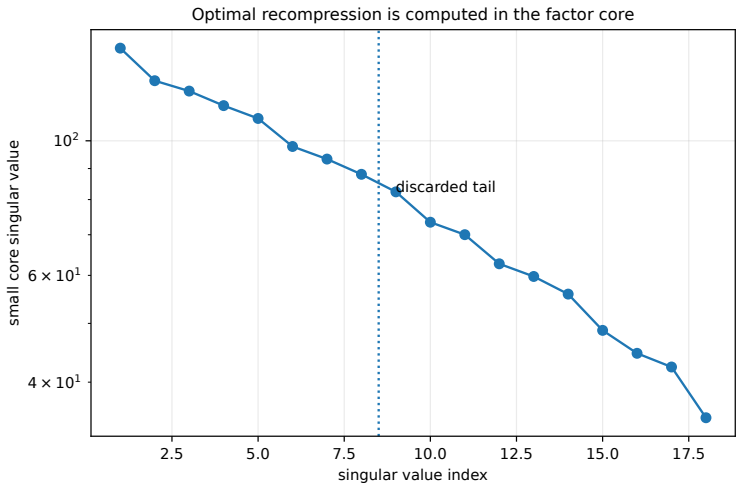


Figure 6. The approximation decision is made from the singular values of a small core whose dimension is the unrecompressed rank.

and let

$$G = W\Sigma Z^*$$

be a singular value decomposition. Define

$$L_k = Q_L W_k \Sigma_k^{1/2}, \qquad R_k = Q_R Z_k \Sigma_k^{1/2}.$$

Then

$$L_k R_k^*$$

is a best rank at most k approximation to LR^*. Its spectral and Frobenius errors are the singular value tails in standard fact 5.5*. The core dimensions equal the column ranks of the two input factors. A noncompact reduced QR may retain extra zero singular directions without changing the conclusion.*

Proof. The exact factor product is

$$LR^* = Q_L G Q_R^*.$$

The two outer maps are isometries on their column spaces. They preserve the nonzero singular values and every unitarily invariant norm of the core. Truncating the singular value decomposition of G and applying standard fact 5.5 gives the result. The compact QR dimensions equal the ranks of the factor column spaces. □

Remark 5.7. The large matrices enter through QR factorizations of the factors. The singular value problem has dimension at most the unrecompressed

rank. Updating and recompressing low rank factors is standard numerical linear algebra [13] and [4].

THEOREM 5.8 (Product chain perturbation). *Let $A_1, \dots, A_m$ be exact factors and let*

$$\widehat{A}_j = A_j + E_j.$$

Then

$$\boxed{\left\| \widehat{A}_m \cdots \widehat{A}_1 - A_m \cdots A_1 \right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \left\| \widehat{A}_i \right\| \right) \|E_j\| \left(\prod_{i=1}^{j-1} \|A_i\| \right).}$$

Empty products are one.

Proof. Use the telescoping identity

$$\widehat{A}_m \cdots \widehat{A}_1 - A_m \cdots A_1 = \sum_{j=1}^{m} \widehat{A}_m \cdots \widehat{A}_{j+1} E_j A_{j-1} \cdots A_1.$$

Take norms term by term. □

THEOREM 5.9 (Certified repeated recompression). *Let*

$$P_0 = I, \qquad P_j = A_j P_{j-1},$$

be exact partial products. Suppose an implemented chain satisfies

$$\widehat{P}_0 = I, \qquad \widehat{P}_j = A_j \widehat{P}_{j-1} + E_j,$$

where E_j is the additive error introduced after the jth multiplication, for example by factor recompression. Then

$$\boxed{\left\| \widehat{P}_m - P_m \right\| \le \sum_{j=1}^{m} \left(\prod_{i=j+1}^{m} \|A_i\| \right) \|E_j\|.}$$

If $\|E_j\| \le \delta_j$, the same formula with δ_j gives a certified final error.

Proof. Set $D_j = \widehat{P}_j - P_j$. Then

$$D_j = A_j D_{j-1} + E_j, \qquad D_0 = 0.$$

Repeated substitution gives

$$D_m = \sum_{j=1}^{m} A_m A_{m-1} \cdots A_{j+1} E_j.$$

Taking norms yields the result. □

6. **Sparse transfer graphs and schedules**

6.1. *Support under algebraic operations.*

THEOREM 6.1 (Path support of powers). *For every positive integer m,*

$$P_\gamma A^m P_\alpha$$

is the sum over directed paths

$$\alpha = \alpha_0 \longrightarrow \alpha_1 \longrightarrow \cdots \longrightarrow \alpha_m = \gamma$$

in G_A, with contribution

$$A_{\alpha_m \alpha_{m-1}} \cdots A_{\alpha_1 \alpha_0}.$$

If no such path exists, the block vanishes.

Proof. Insert the packet identity between every two consecutive factors of A. Every nonzero term determines one path of length m. □

COROLLARY 6.2 (Polynomial support). *If*

$$p(z) = \sum_{j=0}^{d} c_j z^j,$$

then

$$\operatorname{supp}(p(A))$$

is contained in the union of the support relations of A^j for $0 \le j \le d$.

Proof. Apply theorem 6.1 to every power in the polynomial. □

COROLLARY 6.3 (Acyclic transfer nilpotence). *Let K contain only strict transfer blocks. If its support graph is acyclic and its longest directed path has h edges, then*

$$K^{h+1} = 0.$$

Proof. There is no directed path of length $h + 1$. Every block of K^{h+1} therefore vanishes by theorem 6.1. □

6.2. *Block triangular solves.*

Definition 6.4 (Topological level). Let G be a directed acyclic graph. The first level contains its source vertices. Remove them. The next level contains the new sources. Continue until every vertex has been assigned.

THEOREM 6.5 (Level schedule for a triangular solve). *Let $A = D + K$ be block lower triangular after a topological ordering. Assume every diagonal block D_α is invertible. The solve*

$$Ax = b$$

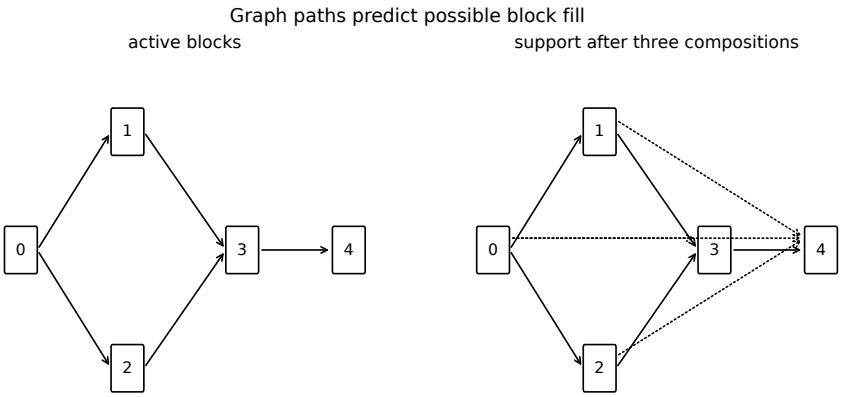


Figure 7. Composition creates possible support along directed paths. Cancellation can remove an allowed block, so the graph gives an inclusion rather than an equality.

can be completed in one sequential stage per topological level. Blocks in the same level may be solved independently after all earlier levels are known.

Proof. The equation in packet β is

$$D_\beta x_\beta = b_\beta - \sum_{\alpha \to \beta} K_{\beta\alpha} x_\alpha.$$

Every predecessor of β lies in an earlier level. Once those values are available, x_β is obtained from the local solve with D_β. Vertices in the same level have no directed edge between them, so their solves are independent. □

COROLLARY 6.6 (Sequential depth). *The number of sequential stages in the level schedule equals one plus the length of the longest directed path.*

Proof. The recursive level of a vertex is one plus the largest level of its predecessors. The largest level therefore counts the vertices on a longest path. □

6.3. *Fill under elimination.*

THEOREM 6.7 (Inverse support lies in the transitive closure). *Let A be an invertible packet operator. If there is no directed path from packet α to packet β in G_A, then*

$$\boxed{P_\beta A^{-1} P_\alpha = 0.}$$

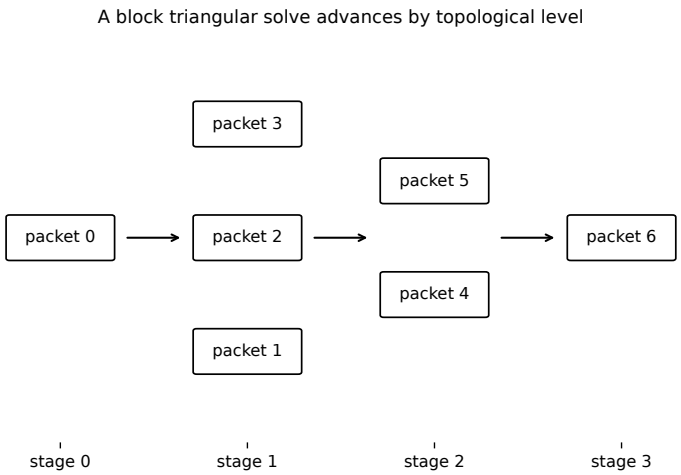


Figure 8. Packets in the same topological level can be processed concurrently in a triangular solve.

Proof. Let R be the set of packets reachable from α, including α, and set

$$X_R = \bigoplus_{\gamma \in R} X_\gamma.$$

No active edge leaves R, so $AX_R \subseteq X_R$. The restriction of A to the finite dimensional space X_R is injective. It is therefore onto X_R. Hence $A^{-1}X_R \subseteq X_R$. Since $\beta \notin R$, the stated block of A^{-1} vanishes. □

Proposition 6.8 (Schur support inclusion). *Split the packets into retained indices R and hidden indices H. Suppose the hidden block A_{HH} is invertible. The retained Schur complement is*

$$S = A_{RR} - A_{RH}A_{HH}^{-1}A_{HR}.$$

A retained block (β, α) can be active only if it was already active in A_{RR}, or if there is a directed walk from α through hidden packets to β.

Proof. The correction block is

$$P_\beta A_{RH}A_{HH}^{-1}A_{HR}P_\alpha.$$

It can be nonzero only when $A_{HR}P_\alpha$ enters a hidden packet, the inverse hidden operator connects hidden packets, and $P_\beta A_{RH}$ exits to β. By theorem 6.7, the hidden inverse can connect only packets joined by a hidden directed path. Cancellation may still remove an allowed block. □

Sparse elimination and fill are treated systematically in [9]. The packet graph records the same structural issue at block scale.

7. **Krylov methods and packet preconditioning**

Definition 7.1 (Krylov space). For a linear operator A, a vector v, and a positive integer m, define

$$\mathcal{K}_m(A, v) = \operatorname{span}\{v, Av, \dots, A^{m-1}v\}.$$

Proposition 7.2 (Invariant packet Krylov decomposition). *Assume*

$$[A, P_\alpha] = 0$$

for every packet. If

$$v = \sum_\alpha v_\alpha, \qquad v_\alpha = P_\alpha v,$$

then

$$A^j v = \sum_\alpha A_\alpha^j v_\alpha$$

for every $j \geq 0$, where

$$A_\alpha = P_\alpha A P_\alpha.$$

Hence

$$\mathcal{K}_m(A, v) \subseteq \bigoplus_\alpha \mathcal{K}_m(A_\alpha, v_\alpha).$$

Proof. Commutation gives

$$A^j P_\alpha = P_\alpha A^j.$$

Apply this identity to each packet component of v. □

Theorem 7.3 (Approximate packet Krylov bound). *Write*

$$A = D + K,$$

where D is packet diagonal. For every positive integer j,

$$\boxed{\left\| A^j - D^j \right\| \leq j M^{j-1} \|K\|,}$$

where

$$M = \max\{\|A\|, \|D\|\}.$$

Consequently,

$$\operatorname{dist}(A^j v, \bigoplus_\alpha \mathcal{K}_{j+1}(D_\alpha, v_\alpha)) \leq j M^{j-1} \|K\| \|v\|.$$

Proof. Use the telescoping identity

$$A^j - D^j = \sum_{k=0}^{j-1} A^{j-1-k} K D^k.$$

Each term has norm at most $M^{j-1}\|K\|$. The vector $D^j v$ lies in the direct sum of local Krylov spaces. □

THEOREM 7.4 (Block Jacobi condition bound). *Let*

$$A = D + K$$

be Hermitian positive definite with positive block diagonal part D. Suppose

$$E = D^{-1/2} K D^{-1/2}$$

is Hermitian and

$$\|E\|_2 \leq \eta < 1.$$

Then

$$\boxed{\operatorname{cond}_2(D^{-1/2} A D^{-1/2}) \leq \frac{1+\eta}{1-\eta}.}$$

Proof. For every unit vector x,

$$|x^* E x| \leq \eta.$$

The Rayleigh quotient of $I + E$ lies in $[1-\eta, 1+\eta]$. The quotient of the extreme eigenvalue bounds gives the result. □

Standard Fact 7.5 (Preconditioned conjugate gradient convergence). Let A and M be Hermitian positive definite, and set

$$\kappa = \operatorname{cond}_2(M^{-1/2} A M^{-1/2}).$$

The energy norm error of preconditioned conjugate gradient satisfies

$$\|x_m - x_*\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^m \|x_0 - x_*\|_A .$$

This standard estimate is discussed in [24]. Combining it with theorem 7.4 yields an explicit iteration certificate when the normalized cross packet coupling is small.

COROLLARY 7.6 (Certified packet preconditioning). *Under the hypotheses of theorem* 7.4, *the convergence factor in standard fact* 7.5 *is bounded by the value obtained from*

$$\kappa = \frac{1+\eta}{1-\eta}.$$

Proof. Insert the condition bound from theorem 7.4 into the monotone convergence factor in standard fact 7.5. □

8. **Packetwise mixed precision**

8.1. *One operator.*

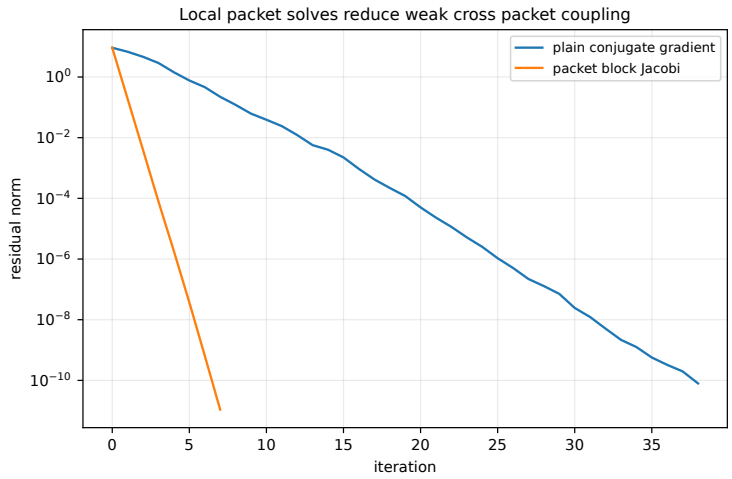


Figure 9. The block diagonal packet operator serves as a reusable local preconditioner in a weakly coupled positive system.

THEOREM 8.1 (Packet perturbation certificate). *Let*

$$A = \sum_{e \in E} A_e, \qquad \widehat{A} = \sum_{e \in E} (A_e + E_e),$$

where edge e starts at packet $\alpha(e)$, and assume

$$A_e = A_e P_{\alpha(e)}, \qquad E_e = E_e P_{\alpha(e)}.$$

Then

$$\boxed{\left\| (\widehat{A} - A)x \right\| \leq \sum_{e \in E} \|E_e\| \left\| P_{\alpha(e)} x \right\|.}$$

If

$$\|E_e\| \leq \varepsilon_e \|A_e\|,$$

then

$$\boxed{\left\| (\widehat{A} - A)x \right\| \leq \sum_{e \in E} \varepsilon_e \|A_e\| \left\| P_{\alpha(e)} x \right\|.}$$

Proof. Exact packet evaluation gives

$$(\widehat{A} - A)x = \sum_{e} E_e P_{\alpha(e)} x.$$

Apply the triangle inequality and the operator norm bound to each term. □

Standard Fact 8.2 (Floating point matrix product model). Under the standard floating point model, a computed matrix product may be written as an exact product with a dimension dependent backward perturbation. The

perturbation constant is controlled by the unit roundoff and the inner dimension when overflow and underflow are excluded.

Detailed forms and their hypotheses are given in [17]. Packetwise use of this fact produces edge bounds ε_e for theorem 8.1.

8.2. *Several operators.*

Corollary 8.3 (Mixed precision product chain). *If each factor A_j is replaced by a packetwise approximation $\widehat{A}_j$, then the final operator error is bounded by theorem* 5.8. *Each $\|E_j\|$ may in turn be bounded by the packet certificate in theorem* 8.1.

Proof. Apply theorem 8.1 at every stage and substitute the resulting operator error bounds into theorem 5.8. □

8.3. *Continuous allocation.*

Theorem 8.4 (Continuous precision allocation). *Let $s_e > 0$ be sensitivities and $c_e > 0$ be costs. Consider*

$$\min_{\varepsilon_e>0} \sum_e c_e \log \frac{1}{\varepsilon_e}$$

subject to

$$\sum_e s_e \varepsilon_e = \tau.$$

The unique minimizer is

$$\boxed{\varepsilon_e = \frac{\tau c_e}{s_e \sum_f c_f}.}$$

Proof. The Lagrangian derivative gives

$$-\frac{c_e}{\varepsilon_e} + \lambda s_e = 0.$$

Thus

$$\varepsilon_e = \frac{c_e}{\lambda s_e}.$$

The constraint gives

$$\lambda = \frac{\sum_f c_f}{\tau}.$$

The objective is strictly convex on the positive orthant, so the stationary point is the unique minimizer. □

Theorem 8.5 (Clipped precision allocation). *Add upper bounds*

$$0 < \varepsilon_e \le m_e.$$

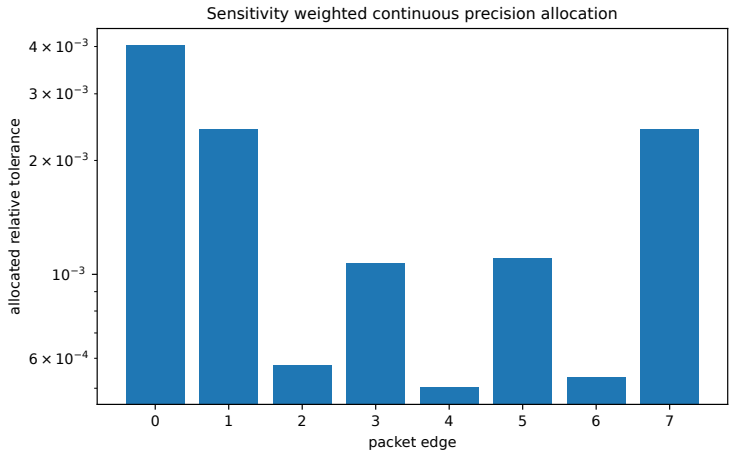


Figure 10. More sensitive packet edges receive tighter continuous tolerances.

Assume

$$0 < \tau \leq \sum_e s_e m_e.$$

There is a unique $\lambda > 0$*, except at the maximal tolerance endpoint, such that*

$$\boxed{\varepsilon_e = \min\left\{m_e, \frac{c_e}{\lambda s_e}\right\}}$$

and

$$\sum_e s_e \varepsilon_e = \tau.$$

This vector minimizes the same objective over the clipped feasible set.

Proof. The Karush Kuhn Tucker conditions give the unconstrained form on every nonactive coordinate and equality with m_e on every active upper bound. The weighted sum

$$\phi(\lambda) = \sum_e s_e \min\left\{m_e, \frac{c_e}{\lambda s_e}\right\}$$

is continuous and nonincreasing. It decreases from the maximal feasible value to zero. The constraint determines λ. Strict convexity gives uniqueness of the minimizer. □

8.4. *Discrete data types.*

Definition 8.6 (Discrete precision option). For edge e, a discrete precision option k has a measured or modeled cost $c_{e,k}$ and a nonnegative integer error weight $w_{e,k}$. Exactly one option must be selected on every edge.

THEOREM 8.7 (Exact discrete precision program). *Let the total integer error budget be B. Define*

$$F_j(b)$$

as the minimum cost for assigning precisions to the first j edges with total weight exactly b. Then

$$F_0(0) = 0, \qquad F_0(b) = +\infty \quad \text{for } b \neq 0.$$

Set $F_{j-1}(u) = +\infty$ whenever $u < 0$ or the state is unreachable. Then

$$\boxed{F_j(b) = \min_k \{F_{j-1}(b - w_{j,k}) + c_{j,k}\}.}$$

The minimum of $F_m(b)$ over $0 \leq b \leq B$ is the exact minimum cost for the quantized model.

Proof. Every feasible assignment for the first j edges ends with one option k on edge j. Removing it leaves an optimal or suboptimal assignment for the first $j-1$ edges with residual weight $b - w_{j,k}$. Replacing a suboptimal prefix by an optimal one can only lower cost. The recurrence therefore has optimal substructure. Induction on j proves the result. □

Remark 8.8. The discrete program is exact only for the supplied options and quantized error weights. Real kernels also depend on accumulation type, scaling, overflow, underflow, tensor core availability, and shape restrictions. Mixed precision iterative refinement has its own conditioning requirements [5].

9. **Discovery and exact packet compilation**

9.1. *Exact spectral discovery.*

Standard Fact 9.1 (Simultaneous spectral decomposition). A finite family of pairwise commuting normal matrices admits a common orthonormal basis of joint eigenvectors. The joint eigenspaces associated with distinct eigenvalue tuples give the coarsest orthogonal packet system on which every matrix in the family acts as a scalar.

PROPOSITION 9.2 (Exact block discovery certificate). *For an orthogonal packet system (P_α), define the block diagonal compression*

$$\Pi_{\mathcal{P}}(A) = \sum_\alpha P_\alpha A P_\alpha.$$

Then

$$\boxed{\|A - \Pi_{\mathcal{P}}(A)\|_{\mathrm{F}}^2 = \sum_{\alpha \neq \beta} \|P_\beta A P_\alpha\|_{\mathrm{F}}^2.}$$

In operator norm,

$$\boxed{\|A - \Pi_{\mathcal{P}}(A)\|_2 \leq \sum_{\alpha \neq \beta} \|P_\beta A P_\alpha\|_2 .}$$

Thus any proposed packet discovery has an exact finite residual certificate after its projectors are known.

Proof. Distinct matrix blocks are orthogonal in the Hilbert Schmidt inner product. Their squared Frobenius norms therefore add. The operator norm estimate follows from the triangle inequality. □

This supplies one exact discovery regime and one exact a posteriori certificate. Perturbation of spectral packets requires a gap. The Davis Kahan theorem gives quantitative control for Hermitian spectral subspaces under such a gap [8]. Randomized low rank discovery supplies another controlled regime [16] and [20]. Neither result gives globally optimal packet discovery for arbitrary operators.

9.2. *Candidate packet trees.*

Definition 9.3 (Candidate representation tree). A candidate representation tree is a finite rooted tree. Every node v has

$$\kappa_v \geq 0, \qquad \epsilon_v \geq 0,$$

representing the cost and error of keeping v as one leaf. Every internal node also has a split overhead $\sigma_v \geq 0$. An admissible pruning contains exactly one selected node on every root to terminal path.

The cost and error of a pruning $\mathcal{L}$ are

$$\operatorname{cost}(\mathcal{L}) = \sum_{v \in \mathcal{L}} \kappa_v + \sum_{u \text{ split}} \sigma_u$$

and

$$\operatorname{err}(\mathcal{L}) = \sum_{v \in \mathcal{L}} \epsilon_v.$$

The model can include arithmetic, storage, launch, transfer, and precision costs inside κ_v. It remains exact only when the declared totals are additive.

THEOREM 9.4 (Exact Lagrangian packet compiler). *Fix* $\lambda \geq 0$ *and define*

$$J_\lambda(\mathcal{L}) = \operatorname{cost}(\mathcal{L}) + \lambda \operatorname{err}(\mathcal{L}).$$

For every node v, *set*

$$K_\lambda(v) = \kappa_v + \lambda \epsilon_v$$

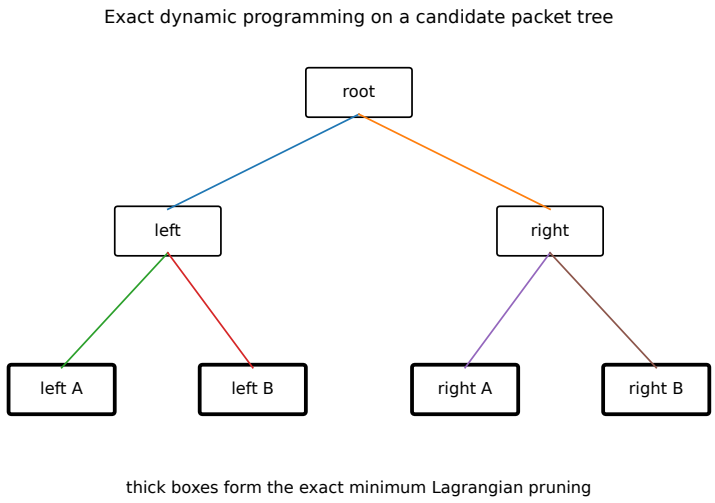


Figure 11. The thick leaves are selected by the exact Lagrangian recurrence.

and

$$S_\lambda(v) = \sigma_v + \sum_{w \in \mathrm{child}(v)} F_\lambda(w).$$

For a terminal node, set

$$F_\lambda(v) = K_\lambda(v).$$

For an internal node, the recurrence

$$\boxed{F_\lambda(v) = \min\{K_\lambda(v), S_\lambda(v)\}}$$

computes the exact minimum Lagrangian objective over every admissible pruning of the subtree rooted at v.

Proof. A pruning of the subtree rooted at v has two possible forms. It keeps v, in which case its objective is $K_\lambda(v)$. Otherwise it splits v and independently chooses an admissible pruning in every child subtree. By induction, the optimal child values are $F_\lambda(w)$. The split objective is therefore $S_\lambda(v)$. Taking the smaller value is exact. □

This recurrence is a computational version of best basis selection and Bellman optimality [3] and [6]. The node costs here may include operator application, transfer, launch, precision, and output conversion rather than only a signal entropy.

Corollary 9.5 (Shared compilation for an operator family). *Let operators $A^{(1)}, \ldots, A^{(m)}$ use the same candidate tree. Give operator j a nonnegative reuse*

weight r_j. Replace the local keep cost and error by

$$\kappa_v = \kappa_v^{\mathrm{setup}} + \sum_{j=1}^{m} r_j \kappa_v^{(j)}$$

and

$$\epsilon_v = \sum_{j=1}^{m} r_j \epsilon_v^{(j)}.$$

Then theorem 9.4 gives the exact optimal shared pruning for the additive weighted objective.

Proof. The weighted family objective is still the sum of local leaf and split terms. Apply theorem 9.4 to the aggregated quantities. □

PROPOSITION 9.6 (Monotone Lagrangian frontier). *Let $0 \le \lambda_1 < \lambda_2$. Choose optimal prunings $\mathcal{L}_1$ and $\mathcal{L}_2$ for the two Lagrangian parameters. Then*

$$\boxed{\operatorname{err}(\mathcal{L}_2) \le \operatorname{err}(\mathcal{L}_1)}$$

and

$$\boxed{\operatorname{cost}(\mathcal{L}_2) \ge \operatorname{cost}(\mathcal{L}_1).}$$

Proof. Optimality gives

$$\operatorname{cost}(\mathcal{L}_1) + \lambda_1 \operatorname{err}(\mathcal{L}_1) \le \operatorname{cost}(\mathcal{L}_2) + \lambda_1 \operatorname{err}(\mathcal{L}_2)$$

and

$$\operatorname{cost}(\mathcal{L}_2) + \lambda_2 \operatorname{err}(\mathcal{L}_2) \le \operatorname{cost}(\mathcal{L}_1) + \lambda_2 \operatorname{err}(\mathcal{L}_1).$$

Adding the inequalities yields

$$(\lambda_2 - \lambda_1)\big(\operatorname{err}(\mathcal{L}_2) - \operatorname{err}(\mathcal{L}_1)\big) \le 0.$$

The error ordering follows. Substitution into the first inequality gives the cost ordering. □

COROLLARY 9.7 (Supported Pareto points). *For $\lambda > 0$, every Lagrangian optimum is Pareto optimal for the pair*

$$(\mathrm{cost}, \mathrm{err}).$$

The quantized budget compiler may recover Pareto points that are not optimal for any Lagrangian parameter.

Proof. If another pruning had no larger cost and no larger error, with one strict inequality, its Lagrangian objective would be smaller. This contradicts optimality. A finite nonconvex frontier can contain points that are not supported by a line of slope $-\lambda$, so the budget formulation is genuinely additional. □

9.3. *An exact error budget compiler.*

THEOREM 9.8 (Quantized budget dynamic program). *Assume every keep error is a nonnegative integer w_v. Let $B \in \mathbb{N}$. For each node v, define $F_v(b)$ as the minimum cost of a pruning of the subtree at v with total error exactly b. The keep option contributes*

$$F_v(w_v) \leq \kappa_v.$$

The split option is obtained by min plus convolution of the child tables, followed by addition of σ_v. The smallest value among

$$F_{\mathrm{root}}(b), \qquad 0 \leq b \leq B,$$

is the exact minimum cost among prunings whose quantized error is at most B.

Proof. The keep case is immediate. In the split case, an admissible pruning is the union of one admissible pruning from each child. Its error is the sum of the child errors and its cost is the sum of their costs plus σ_v. Repeated min plus convolution enumerates every possible distribution of the integer budget across the child subtrees. Induction on the tree proves exactness. □

COROLLARY 9.9 (Pseudo polynomial complexity). *If the maximum number of children is bounded and the budget is B, the quantized compiler has pseudo polynomial complexity in the number of tree nodes and B. A direct pairwise convolution implementation has quadratic dependence on B at each merge.*

Proof. Every table has at most $B+1$ entries. Merging two tables by direct convolution examines at most $(B+1)^2$ pairs. A bounded number of merges occurs at every node. □

9.4. *Decision stability and local updates.*

Definition 9.10 (Decision gap). At an internal node v, let

$$g_v = |K_\lambda(v) - S_\lambda(v)|.$$

This is the gap between the keep and split scores after the child subproblems have been solved.

THEOREM 9.11 (Compiler stability). *Suppose the keep score and split score at every internal node change by less than $g_v/2$. Then every strict keep or split decision in the original optimal pruning remains unchanged.*

Proof. If the original keep score is lower by g_v, then after perturbation it can increase by less than $g_v/2$, while the split score can decrease by less than $g_v/2$. The order remains strict. The split case is the same. Apply the argument from the leaves upward. □

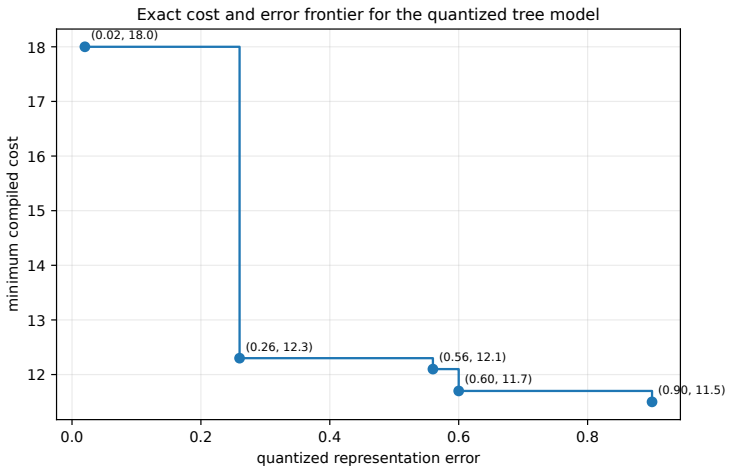


Figure 12. The quantized dynamic program produces the exact cost and error frontier for its finite model.

THEOREM 9.12 (Local recompilation). *Suppose local costs change only inside the subtree rooted at u. Then every dynamic programming value outside that subtree and outside the ancestor chain from u to the root remains unchanged. Recomputing the subtree at u and that ancestor chain gives the exact new global optimum.*

Proof. A node value depends only on its own local data and the values of its children. Nodes outside the changed subtree and outside the ancestor chain have the same local data and the same child values. Their values remain unchanged. Every affected dependency lies inside the changed subtree or on the unique path from u to the root. □

Remark 9.13 (Boundary of the compiler theorem). The exact compiler requires a finite candidate tree and additive local models. Arbitrary overlapping decompositions, nonlocal cache interactions, global rank coupling, and unrestricted structure discovery can destroy optimal substructure. No universal approximation factor is proved for those settings.

10. **Device scheduling and amortization**

10.1. *Shape classes.*

Definition 10.1 (Shape class). Two active packet blocks lie in the same shape class when their input dimension, output dimension, right hand side count, scalar type, accumulation type, and kernel contract agree.

PROPOSITION 10.2 (Launch reduction by shape). *Suppose B independent active blocks fall into g shape classes, and suppose the backend accepts one batched array or pointer list for each class. Grouped execution then requires at most g launches instead of B, without changing the arithmetic or the output.*

Proof. Place all blocks in one class along a batch dimension and invoke one batched kernel for that class. Every block receives the same local computation. One launch per class is sufficient. □

CUDA provides graph capture and replay for repeated dependency graphs [22]. These mechanisms reduce repeated launch work, but they do not change the operator or its arithmetic count.

10.2. *Critical path.*

THEOREM 10.3 (Acyclic schedule depth). *Let a packet task graph be acyclic. Assume every task takes one normalized stage, every edge is a dependency, and an unlimited number of independent tasks may run concurrently. The minimum possible number of sequential stages equals the number of vertices on a longest directed path.*

Proof. Every path gives a lower bound because its tasks must execute in order. The topological level schedule in theorem 6.5 achieves exactly this depth. □

10.3. *One time costs.*

THEOREM 10.4 (Amortization threshold). *Let $s \geq 0$ be a one time setup cost. Let t_b be the baseline time per use and t_c the candidate time per use. If $t_b > t_c$, the candidate is faster after R uses exactly when*

$$R > \frac{s}{t_b - t_c}.$$

The smallest integer reuse count is

$$\boxed{R_{\min} = \left\lfloor \frac{s}{t_b - t_c} \right\rfloor + 1.}$$

If $t_b \leq t_c$, no positive reuse count amortizes a positive setup cost.

Proof. Compare

$$s + Rt_c$$

with

$$Rt_b.$$

Rearranging gives the stated inequality. □

COROLLARY 10.5 (Graph replay threshold). *If graph capture costs c, ordinary execution costs t_o, and replay costs $t_r < t_o$, then capture is profitable when*

$$R > \frac{c}{t_o - t_r}.$$

Proof. Apply theorem 10.4 with setup cost c. □

COROLLARY 10.6 (Materialization threshold). *Suppose a compressed representation costs t_c per downstream use, an explicit representation costs t_e per use, and one time materialization costs m. Materialization is profitable only when*

$$R > \frac{m}{t_c - t_e}$$

and $t_c > t_e$.

Proof. Compare repeated compressed use with materialization followed by repeated explicit use. □

11. **Benchmark contracts and measurement protocol**

Performance data are meaningful only after the mathematical and timing contracts are fixed. The protocol follows standard concerns from numerical linear algebra, floating point analysis, and machine performance modeling [17], [14], and [26].

Definition 11.1 (Case signature). A case signature records the dimensions, packet layout, rank, right hand side count, random seed, source values, source type, reference computation, and output contract.

Definition 11.2 (Fair match). Two measured rows are a fair match when they share one case signature, return the same declared output, use the same correctness reference, and have identical timing scopes. If one row includes an extra conversion or setup cost, it is a different scope and must be reported separately or added to both rows before comparison.

PROPOSITION 11.3 (Fair matching is an equivalence relation). *On rows with complete metadata, equality of the case signature, output contract, correctness reference, and timing scope is an equivalence relation.*

Proof. Equality of each metadata field is reflexive, symmetric, and transitive. Their conjunction has the same properties. □

Definition 11.4 (Representation gain). Representation gain compares dense and structured work or storage for the same operator, independently of the processor.

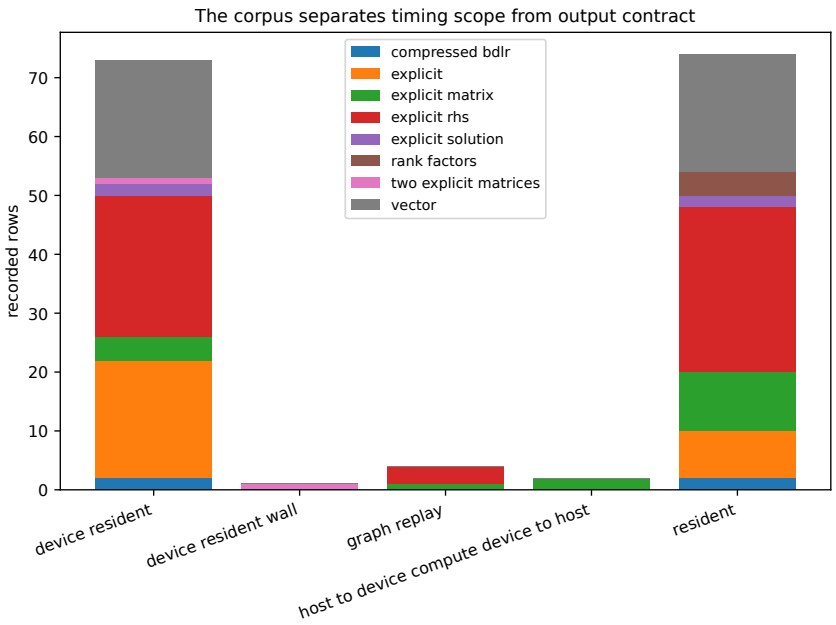


Figure 13. The corpus records timing scope and output contract as separate fields.

Definition 11.5 (Device gain). Device gain compares processors for the same representation, output, and timing scope.

Definition 11.6 (Kernel gain). Kernel gain compares two implementations on the same device for the same representation, output, and timing scope.

Definition 11.7 (Amortization gain). Amortization gain records the effect of one time discovery, compilation, transfer, capture, or materialization at a declared reuse count.

A measured total speedup can mix these four effects. A multiplicative decomposition is only exact when matched rows isolate the factors. Otherwise the decomposition is interpretive.

Remark 11.8 (Logical work rate). A structured method may report a dense equivalent logical work rate greater than the physical dense throughput because it does not execute the dense operations. This statistic describes avoided work. It is not a claim that the device performed those operations.

12. **Recorded CPU and GPU corpus**

The empirical corpus contains 154 measured rows from sixteen suites. It has 74 CPU rows and 80 GPU rows. The GPU was an NVIDIA GeForce RTX

Suite	Rows	Suite	Rows
Dense multiplication	28	BDLR application	15
Block diagonal	12	BDLR product	12
Circulant	12	Toeplitz	12
Kronecker	12	Block banded	8
Sparse plus low rank	8	HODLR	8
Strassen	6	Transfer pipeline	5
Walsh Hadamard	4	Hidden blocks	4
Randomized low rank	4	Iterative refinement	4

Table 1. Coverage of the sanitized measured corpus.

Method	Median time in ms	Logical GFLOP per second	Relative error
FP64	541.2803	253.91	2.18×10^{-15}
FP32	18.1309	7580.35	5.76×10^{-7}
TF32	8.6999	15797.75	2.94×10^{-4}
FP16	4.0264	34134.72	3.60×10^{-4}
BF16	4.1482	33132.00	2.88×10^{-3}

Table 2. Recorded dense accuracy and throughput at one dimension.

3070 Ti Laptop GPU with compute capability 8.6 and 8 GB nominal memory. The recorded software used PyTorch 2.11.0 with a CUDA 12.8 build. These facts identify the environment. They do not predict another machine.

12.1. *Dense precision frontier.* At dimension 4096, the device resident dense rows gave the values in table 2.

The frontier is not a universal ordering of scalar types. It records one generated family. Packet precision assignment should use local sensitivities and supported kernels rather than one global type.

12.2. *Regular packet operators.* For an 8192 by 8192 equal block operator with block width 64 and 32 right hand sides, the batched packet calculation took

$$0.046032 \text{ ms.}$$

It was

$$14.8919$$

times faster than the matched device resident dense GPU row and

$$235.7404$$

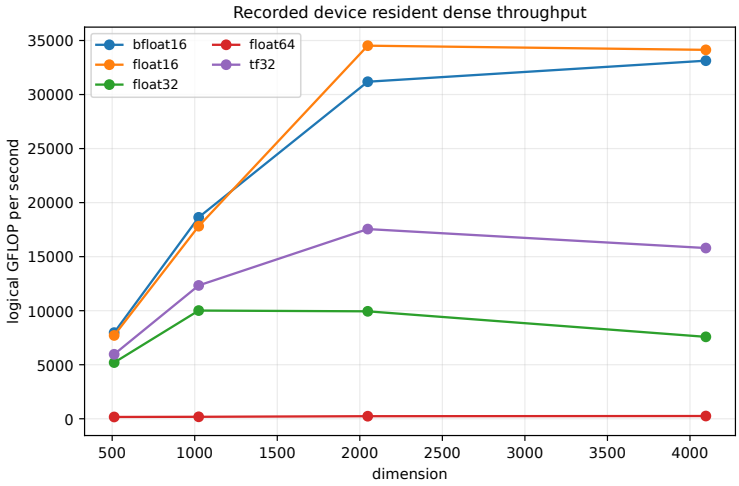


Figure 14. Recorded device resident dense throughput across dimensions and scalar types.

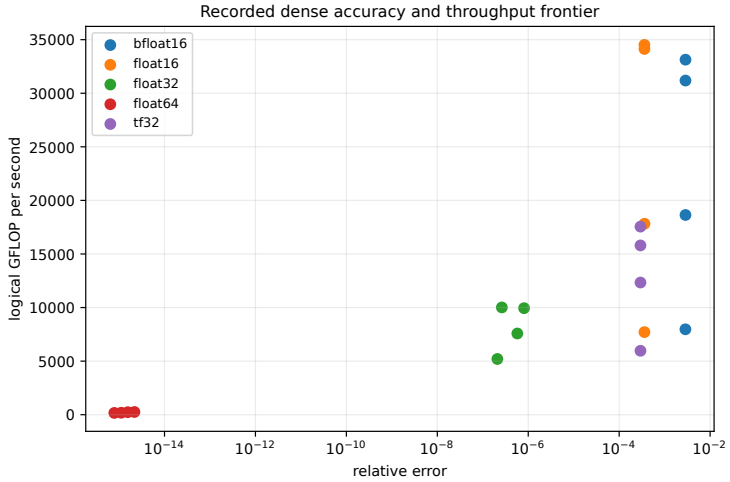


Figure 15. The speed and error tradeoff depends on both precision and problem size.

times faster than the matched CPU dense row. The relative error was

$$8.17 \times 10^{-8}.$$

For BDLR application with the same dimension and right hand side count, block width 64, and rank 8, CUDA Graph replay took

$$0.079872 \text{ ms}.$$

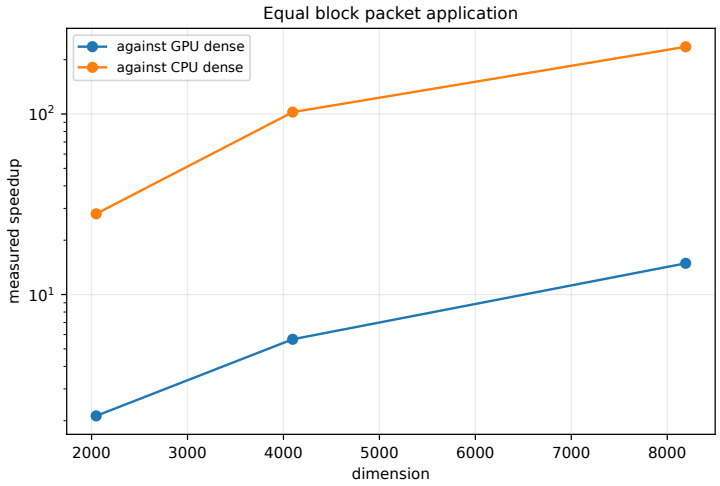


Figure 16. Regular equal shapes expose both representation reduction and batched execution.

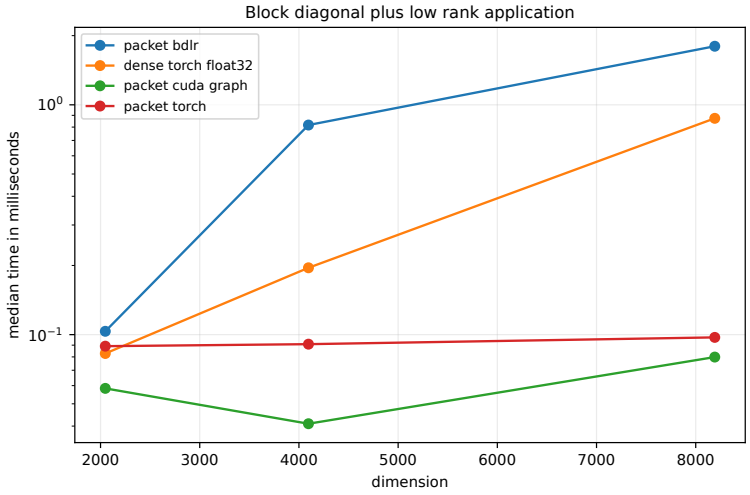


Figure 17. Representation and launch strategy both affect BDLR application time.

It was

$$10.9355$$

times faster than the matched dense GPU row and

$$153.6527$$

times faster than the matched CPU dense row. Capture time was excluded from the replay scope and must be amortized separately.

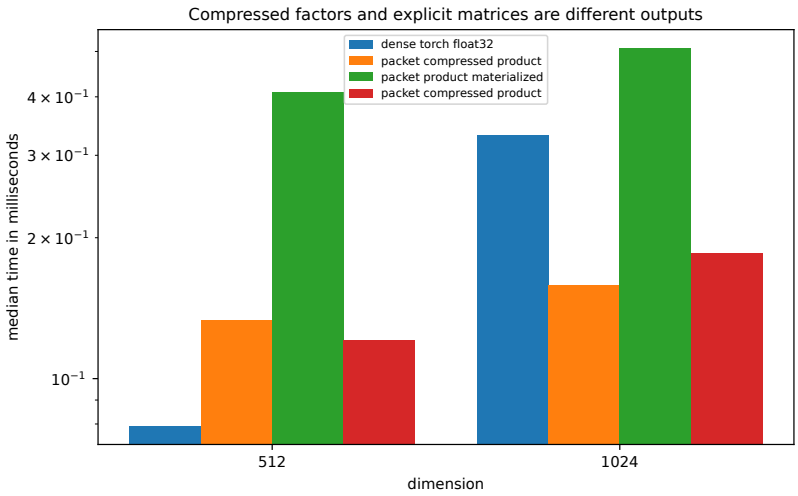


Figure 18. Compressed and explicit products answer different computational questions.

12.3. *Compressed products.* A compressed BDLR product returns block and low rank factors. Materialization returns all matrix entries. The measured product rows show the cost of changing the output contract.

The algebra in theorem 5.1 explains why the compressed product can remain small. The materialization cost in proposition 3.10 explains why explicit output can erase the advantage.

12.4. *Discovery and transfer.* For hidden permuted blocks at dimension 512, structure discovery took 2.1102 ms, dense application took 0.0969 ms, and packet application took 0.0151 ms. The exact threshold in theorem 10.4 is 26 uses. At dimension 1024, the corresponding values were 5.2726, 0.1896, and 0.0241 ms, giving 32 uses.

For one 2048 dense FP32 product, device resident graph replay took 1.9993 ms. The end to end pageable path took 7.3036 ms. The pinned path took 6.3084 ms. Two serial device resident products took 3.6557 ms, while the tested two stream path took 4.2368 ms.

12.5. *Negative controls.* The corpus retains methods that lost to their matched baselines.

- At dimension 512, recursive Strassen multiplication with leaf size 128 took 7.0950 ms, while the matched OpenBLAS dense multiplication took 1.9597 ms.
- At dimension 2048, the tested recursive HODLR GPU path took 5.6079 ms, while dense GPU multiplication took 0.0860 ms.

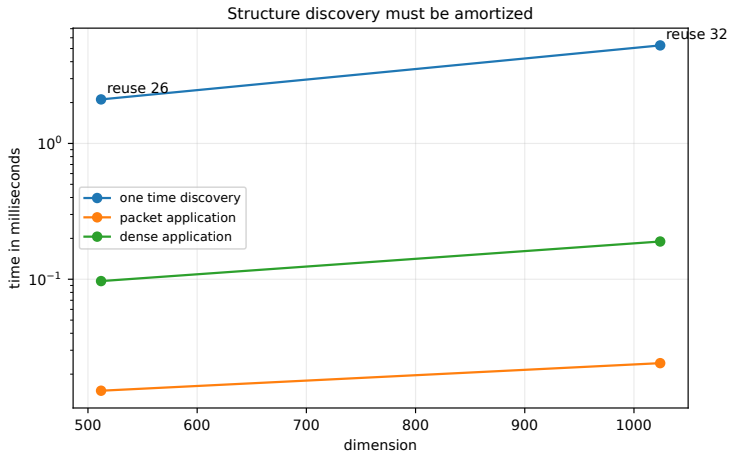


Figure 19. A profitable repeated representation can remain unprofitable for one use.

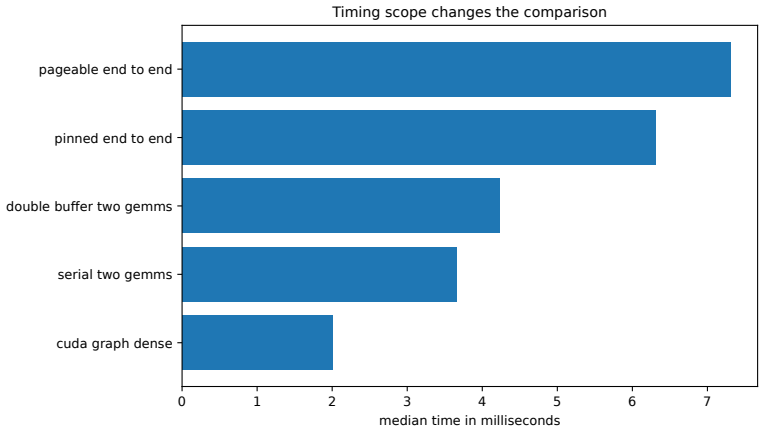


Figure 20. Device residence, transfer, graph replay, and a multioutput pipeline have different timing scopes.

- The tested FP32 iterative refinement solve took 22.0848 ms, while the direct FP64 GPU solve took 12.6286 ms with a comparable final error.
- The two stream dense path was slower than the serial two product path.
- The GPU Walsh Hadamard transform was slower than the CPU implementation at length 65536, then became faster at length 262144.

Fast multiplication can be asymptotically valuable and still lose at measured sizes because of recursion, memory traffic, and kernel granularity. Hierarchical matrix methods have similar crossover issues [1]. Transform algorithms and tensor factorizations form additional structure classes [7] and [23].

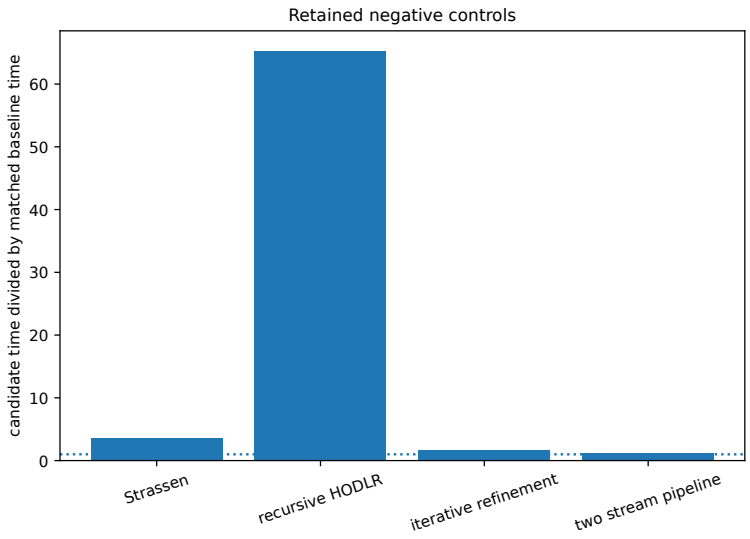


Figure 21. Values above one are slowdowns against the matched baseline.

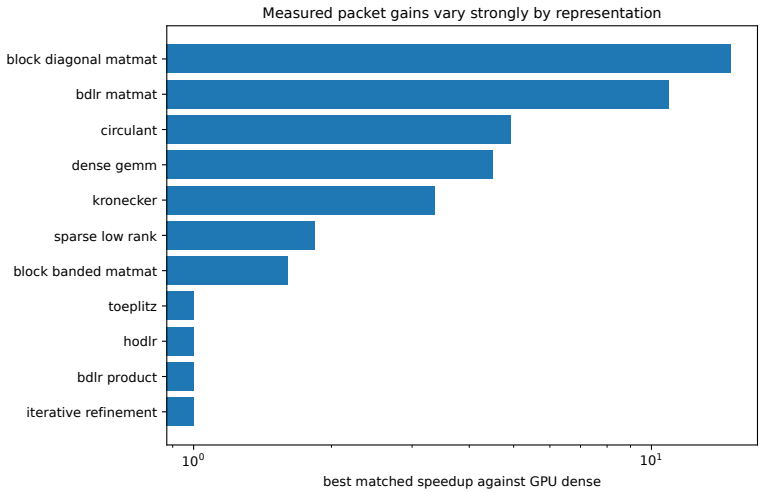


Figure 22. The best matched gain differs strongly across suites. Regular packet structure dominates the largest measured rows.

13. **Implementation and verification**

The release contains a small reference implementation of the exact operators and optimization algorithms used in the paper. The mathematical interface is deliberately narrow.

(1) A packet layout records dimensions and coordinate slices.

Layer	Verification target
Packet semantics	materialization, application, adjoint, composition
Low rank algebra	exact product, Woodbury solve, optimal core recompression
Graph algorithms	path support, topological levels, Schur fill inclusion
Optimization	Lagrangian tree compiler, budget compiler, precision assignment
Numerical bounds	product chain error, block perturbation, clipped allocation
Empirical corpus	row count, suite count, device split, status, error ceiling
Document	proofs, labels, citations, figures, font embedding, render audit

Table 3. Verification layers in the strengthened release.

(2) A packet operator stores active blocks and evaluates them without materializing inactive blocks.
(3) BDLR factors support exact application, product, solve, and small core recompression.
(4) Graph utilities compute support products, path closure, Schur fill inclusion, and topological levels.
(5) Tree dynamic programs solve the Lagrangian and quantized budget models.
(6) A multiple choice dynamic program assigns discrete precision options.
(7) Benchmark utilities audit case identity, output contracts, timing scopes, and retained negative controls.

The full deterministic verifier performs 37392 checks in nine groups. It covers packet evaluation, adjoints, composition, structured primitives, recompression optimality, product chain error, graph support, Schur fill, tree compilation, precision assignment, and the benchmark corpus. The strengthened unit suite contains eleven passing tests. The original source suite contains twenty three passing tests for BDLR operations, Krylov methods, mixed precision, measured data schema, and figure generation.

14. **Hardware implications and exact boundaries**

The finite results suggest a device interface with grouped block kernels, low rank contractions, packet metadata, a transfer graph scheduler, and per edge

precision tags. Such an interface may reduce materialization and repeated launch work when the operator family remains closed under the chosen representation.

The exact results stop at a finite candidate tree and a declared device model. Three broader problems remain open.

(1) A universal discovery algorithm would need to find a profitable packet structure from an arbitrary operator and data distribution. The simultaneous spectral theorem and randomized low rank approximation cover special cases only.
(2) A universal compiler would need a controlled approximation factor when costs are nonlocal and candidate decompositions overlap. The dynamic programs in theorems 9.4 and 9.8 do not cover that setting.
(3) A packet native memory hierarchy would need a communication theorem for a family of operators and a specified architecture. The one pass byte model in proposition 3.6 is not such a lower bound.

Communication lower bounds require explicit machine models [18] and [2]. CUDA Graph behavior and kernel constraints depend on the installed CUDA platform [22]. The measured results in this paper do not prove an optimal accelerator.

The article also makes no claim about the exponent of generic dense matrix multiplication. Packet algorithms answer a different question. They exploit a restricted operator class and may return a compressed output.

15. **Conclusion**

A packet operator representation records which block maps exist, how they are stored, and what output they denote. Exact block evaluation and composition turn the support graph into algebra. Arithmetic, storage, and one pass data movement follow from the active block dimensions. Block diagonal plus low rank operators remain closed under multiplication before recompression. Their inverse is reduced by Woodbury, and their correction can be recompressed optimally through a small singular value problem.

Sparse transfer graphs predict path support, triangular schedule depth, and possible fill under elimination. Packet diagonal structure gives local Krylov spaces and a certified block Jacobi condition bound. Packetwise perturbations give a direct mixed precision certificate. Continuous tolerances follow from a convex allocation, while finite data type choices lead to an exact multiple choice budget program.

A finite candidate refinement tree supports a genuine packet compiler. The Lagrangian optimum, the quantized error budget optimum, and local recompilation are exact under additive hypotheses. This is narrower than

unrestricted automatic discovery, but it turns one broad compiler conjecture into a proved algorithmic statement.

The measured corpus shows the practical distinction between regular and irregular structure. Equal block and BDLR operators gained substantially on the recorded GPU. Recursive HODLR, Strassen, iterative refinement, a small transform, and an attempted overlap lost to matched baselines. Discovery, transfer, capture, and materialization changed the break even point.

The computational principle is therefore precise. Remove work through a verified representation. Preserve the declared output. Compile only inside a model whose costs and errors are stated. Assign precision by sensitivity. Amortize one time costs. Keep negative controls. Treat the final timing as a property of the representation, device, kernel, and reuse regime together.

Language model contribution

An OpenAI language model was used to help reorganize the original manuscript, test alternative theorem formulations, draft portions of the exposition, and design finite verification calculations. Its mathematical suggestions affected the packet intermediate representation, the small core recompression theorem, the product chain error bound, the quantized precision program, the exact tree compiler, the local recompilation theorem, and the separation of benchmark contracts. The author is responsible for checking every statement, proof, citation, computation, and measured claim in any submitted version. The model is not an author.

References

1. Sivaram Ambikasaran and Eric Darve, *An o n log n fast direct solver for partial hierarchically semi separable matrices*, Journal of Scientific Computing **57** (2013), no. 3, 477–501.
2. Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz, *Minimizing communication in numerical linear algebra*, SIAM Journal on Matrix Analysis and Applications **32** (2011), no. 3, 866–901.
3. Richard Bellman, *Dynamic programming*, Princeton University Press, Princeton, NJ, 1957.
4. James R. Bunch and Christopher P. Nielsen, *Updating the singular value decomposition*, Numerische Mathematik **31** (1978), 111–129.
5. Erin Carson and Nicholas J. Higham, *Accelerating the solution of linear systems by iterative refinement in three precisions*, SIAM Journal on Scientific Computing **40** (2018), no. 2, A817–A847.
6. Ronald R. Coifman and Mladen Victor Wickerhauser, *Entropy based algorithms for best basis selection*, IEEE Transactions on Information Theory **38** (1992), no. 2, 713–718.

7. James W. Cooley and John W. Tukey, *An algorithm for the machine calculation of complex fourier series*, Mathematics of Computation **19** (1965), no. 90, 297–301.
8. Chandler Davis and William M. Kahan, *The rotation of eigenvectors by a perturbation. iii*, SIAM Journal on Numerical Analysis **7** (1970), no. 1, 1–46.
9. Timothy A. Davis, *Direct methods for sparse linear systems*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2006.
10. James Demmel, Ioana Dumitriu, and Olga Holtz, *Fast linear algebra is stable*, Numerische Mathematik **108** (2007), no. 1, 59–91.
11. Carl Eckart and Gale Young, *The approximation of one matrix by another of lower rank*, Psychometrika **1** (1936), no. 3, 211–218.
12. Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran, *Cache oblivious algorithms*, Proceedings of the Fortieth Annual Symposium on Foundations of Computer Science, IEEE Computer Society, 1999, pp. 285–297.
13. Gene H. Golub and William Kahan, *Calculating the singular values and pseudo inverse of a matrix*, Journal of the Society for Industrial and Applied Mathematics, Series B: Numerical Analysis **2** (1965), no. 2, 205–224.
14. Gene H. Golub and Charles F. Van Loan, *Matrix computations*, fourth ed., Johns Hopkins University Press, Baltimore, MD, 2013.
15. William W. Hager, *Updating the inverse of a matrix*, SIAM Review **31** (1989), no. 2, 221–239.
16. Nathan Halko, Per Gunnar Martinsson, and Joel A. Tropp, *Finding structure with randomness*: *Probabilistic algorithms for constructing approximate matrix decompositions*, SIAM Review **53** (2011), no. 2, 217–288.
17. Nicholas J. Higham, *Accuracy and stability of numerical algorithms*, second ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 2002.
18. Jia Wei Hong and H. T. Kung, *I o complexity*: *The red blue pebble game*, Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing, Association for Computing Machinery, 1981, pp. 326–333.
19. IEEE Standards Association, *Ieee standard for floating point arithmetic*, IEEE Std 754 2019, 2019.
20. Per Gunnar Martinsson and Joel A. Tropp, *Randomized numerical linear algebra*: *Foundations and algorithms*, Acta Numerica **29** (2020), 403–572.
21. Leon Mirsky, *Symmetric gauge functions and unitarily invariant norms*, The Quarterly Journal of Mathematics **11** (1960), 50–59.
22. NVIDIA Corporation, *Cuda programming guide*, NVIDIA Corporation, 2026, Version consulted May 2026.
23. Ivan V. Oseledets, *Tensor train decomposition*, SIAM Journal on Scientific Computing **33** (2011), no. 5, 2295–2317.
24. Yousef Saad, *Iterative methods for sparse linear systems*, second ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 2003.
25. Volker Strassen, *Gaussian elimination is not optimal*, Numerische Mathematik **13** (1969), 354–356.

26. Samuel Williams, Andrew Waterman, and David Patterson, *Roofline*: *An insightful visual performance model for multicore architectures*, Communications of the ACM **52** (2009), no. 4, 65–76.

Komorebi P.